Universidad del Rosario

Algoritmos y Estructura de Datos ·

Simulacro de preparación — Segundo corte

BLOQUE A — Recursión y backtracking

Ejercicio 1 · Trazado de recursión · Python · (comprensión + trazado)

Dado el siguiente código, determine: **(a)** qué imprime, **(b)** cuántos marcos (frames) llega a tener la pila de llamadas en su punto más profundo, y **(c)** qué ocurre si se elimina la primera condición.

recoger.py — Python 3

```
def recoger(n, k):
    if n == 0:
        return 0
    if n % 2 == 0:
        return k + recoger(n - 1, k + 1)
    return recoger(n - 1, k)

print(recoger(5, 1))
```

Ejercicio 2 · Detección de errores en recursión · C++ · (depuración)

La función debe devolver cuántos elementos de v[0..n-1] son mayores o iguales que umbral. Se invoca con contar(v, v.size(), 20) sobre v = {12, 45, 7, 38, 60, 19, 23}. Identifique los dos defectos, explique por qué el programa es incorrecto y escriba la versión corregida.

contar.cpp — C++17

```
#include <iostream>
#include <vector>
using namespace std;

int contar(const vector<int>& v, int n, int umbral) {
    if (n < 0) return 0;                          //  (1)
    if (v[n] >= umbral)                           //  (2)
        return 1 + contar(v, n - 1, umbral);
    return contar(v, n - 1, umbral);
}
```

BLOQUE B — Algoritmos de búsqueda

Ejercicio 3 · Trazado de búsqueda binaria · C++ · (trazado)

Sobre el arreglo ordenado de códigos, complete la tabla de iteraciones para buscar 175 y para buscar 160. Indique en cada caso cuántas iteraciones ejecuta el ciclo.

buscar.cpp — C++17

```
int v[] = {103, 118, 127, 134, 149, 156, 168, 175, 182, 190, 203, 219};
//  idx:     0    1    2    3    4    5    6    7    8    9   10   11

int buscar(int v[], int n, int x) {
    int izq = 0, der = n - 1;
    while (izq <= der) {
        int medio = (izq + der) / 2;
        if (v[medio] == x) return medio;
        if (v[medio] <  x) izq = medio + 1;
        else               der = medio - 1;
    }
    return -1;
}
```

Ejercicio 4 · Detección de errores en búsqueda binaria · Python · (depuración)

Esta función falla en producción: con algunas entradas no termina nunca. Identifique el defecto, explique por qué se produce el bucle infinito, corríjalo e indique la precondición que debe cumplir v.

buscar.py — Python 3

```
def buscar(v, x):
    izq, der = 0, len(v) - 1
    while izq <= der:
        medio = (izq + der) // 2
        if v[medio] == x:
            return medio
        elif v[medio] < x:
            izq = medio          # (1)
        else:
            der = medio          # (2)
    return -1

v = [8, 15, 23, 31, 42, 56, 64, 77, 89, 91]
```

BLOQUE C — Algoritmos de ordenamiento

Ejercicio 5 · Trazado de Insertion Sort · Python · (trazado)

Muestre el contenido del arreglo después de cada pasada del ciclo externo y cuente el total de comparaciones que realiza el ciclo interno.

insercion.py — Python 3

```
v = [42, 17, 93, 8, 55, 31]

for i in range(1, len(v)):
    actual = v[i]
    j = i - 1
    while j >= 0 and v[j] > actual:
        v[j + 1] = v[j]
        j -= 1
    v[j + 1] = actual
```

Ejercicio 6 · Escritura de código · C++ · (implementación)

Escriba la función fusionar, que recibe dos vectores ya ordenados de forma no decreciente y devuelve un vector nuevo con todos sus elementos ordenados. Es el paso de combinación de Merge Sort. No use sort ni ningún algoritmo de ordenamiento: aproveche que las entradas ya están ordenadas.

Firma y caso de prueba

```
vector<int> fusionar(const vector<int>& a, const vector<int>& b);

// Caso de prueba:
//   a = {5, 18, 27, 40}   b = {9, 18, 33}
//   resultado esperado: 5 9 18 18 27 33 40   (tamaño 7)
```

BLOQUE D — Análisis de complejidad

Ejercicio 7 · Conteo de operaciones y Big O · C++ y Python · (análisis)

Para cada fragmento, escriba la expresión exacta del número de veces que se ejecuta c++ (o c += 1) en función de n, y la complejidad asintótica en notación Big O.

Fragmento A

```
// Fragmento A  —  C++
int f(int n) {
    int c = 0;
    for (int i = 0; i < n; i++)
        for (int j = i; j < n; j++)
            c++;
    return c;
}
```

Fragmento B

```
# Fragmento B  —  Python 3
def g(n):
    c = 0
    i = 1
    while i < n:
        for j in range(n):
            c += 1
        i *= 2
    return c
```

Ejercicio 8 · Comparación de alternativas · Python · (razonamiento + complejidad)

Las dos funciones cuentan cuántos elementos de la lista ya habían aparecido antes. Indique la complejidad de cada una, explique de dónde sale la diferencia y estime cuál será el factor de diferencia con n = 8000.

duplicados.py — Python 3

```
def dup_lista(datos):
    vistos = []                      # lista
    d = 0
    for x in datos:
        if x in vistos: d += 1
        else: vistos.append(x)
    return d

def dup_conjunto(datos):
    vistos = set()                   # conjunto
    d = 0
    for x in datos:
        if x in vistos: d += 1
        else: vistos.add(x)
    return d
```

BLOQUE E — Pilas y colas

Ejercicio 9 · Trazado de cola circular · C++ · (trazado + índices)

Una cola circular con capacidad 5 arranca con frente = 0, final = −1 y cantidad = 0. El método encolar hace final = (final + 1) % 5 y desencolar hace frente = (frente + 1) % 5. Complete el estado de los tres atributos después de cada operación e indique en qué operación se produce el desbordamiento.

Secuencia de operaciones · CAP = 5

```
encolar(11);  encolar(22);  encolar(33);  encolar(44);
desencolar(); desencolar();
encolar(55);  encolar(66);  encolar(77);
encolar(88);
```

Ejercicio 10 · Escritura de código · Python · (implementación con pila)

Escriba la función balanceada(s) que devuelve True si los paréntesis, corchetes y llaves de la cadena s están correctamente balanceados y anidados, y False en caso contrario. Los demás caracteres se ignoran. Use una pila.

Casos de prueba

```
balanceada("(a[b]{c})")  ->  ?
balanceada("([)]")       ->  ?
balanceada("((")         ->  ?
balanceada("")           ->  ?
balanceada("a)b(")       ->  ?
```

BLOQUE F — Conjuntos, mapas y tablas hash

Ejercicio 11 · Mapas y conjuntos · Python · (caso práctico)

A partir de los registros de recolección, escriba el código que acumule los kilos por material, determine el material con mayor total y liste los materiales distintos. Indique qué imprime y explique por qué para una de las dos tareas conviene un dict y para la otra un set.

Datos de entrada

```
registros = [("vidrio", 12), ("papel", 8),  ("vidrio", 5),  ("plastico", 20),
             ("papel", 14),  ("metal", 3), ("plastico", 7), ("vidrio", 9)]
```

Ejercicio 12 · Tabla hash con encadenamiento · C++ · (caso práctico + análisis)

Una tabla hash de M = 7 buckets usa h(k) = k % 7 y resuelve colisiones por encadenamiento. Se insertan, en ese orden, las claves 23, 15, 42, 9, 36, 51, 28, 17. Muestre el contenido de cada bucket, cuente las colisiones, indique la longitud de la cadena más larga y calcule el factor de carga.

k	23	15	42	9	36	51	28	17
h(k) = k % 7								